

08 测试与质量保障

让每一次科学计算，都发生在离思想最近的地方

测试被视为代码库的一等公民：纯逻辑在 Node 环境用 Vitest 做单元测试，产品行为用 Playwright-core 驱动无头浏览器对生产预览做端到端验证，性能与体积则由基线与预算在 CI 中把守。



官网

snowleopard-io.github.io/ErgalicsStudio



GitHub

github.com/SnowLeopard-io/ErgalicsStudio



Gitee

gitee.com/cnt-code/ergalics-studio

本篇章节

一 单元测试

二 端到端测试

三 性能基准与体积
预算

四 持续集成

五 工程脚本

六 质量原则

目 录

一、单元测试 (Vitest)	03
二、端到端测试 (Playwright-core)	04
三、性能基准与体积预算	06
四、持续集成 (5 条 workflows)	06
五、工程脚本	07
六、质量原则	08

测试被视为代码库的一等公民：纯逻辑在 Node 环境用 Vitest 做单元测试，产品行为用 Playwright-core 驱动无头浏览器对生产预览做端到端验证，性能与体积则由基线与预算在 CI 中把守。

当前基线：

指标	数值
单元测试文件	123 个 (tests/ 下共 125 个 .ts，其中 2 个是共享辅助模块)
单元测试用例	2233 个 (2231 通过，2 个在无 GPU 的 CI 上跳过)
端到端套件	13 套 (1 个冒烟脚本 + 12 个 verify-* 脚本)
CI 工作流	5 条 (ci、bench、security、release、deploy)
类型检查	tsc --noEmit 严格模式，零发布；同时充当 lint (npm run lint 即 typecheck)

一 单元测试 (Vitest)

运行方式：npm test (即 vitest run) 执行全部单测；npm run verify 先做类型检查再跑单测，是贡献指引中要求保持绿色的组合检查。

测试文件按主题组织在 tests/ 下，根目录与子目录各占一半：

主题	文件数	覆盖内容
插件运行时与插件族	30 (根目录)	内置插件、科研/地理物理/数据插件、结构力学、三维可视化、插件缺陷回归、沙箱与 cspkg、文件格式路由、插件市场生命周期、i18n、可访问性与对比度、引用元数据、PWA、主题包、存储与运行记录
区块系统	12	数据表运算、区块目录、编译器、执行器、几何、统计区块、文件数据、操作符、可视化桥接
编辑器与 IR	12	共享 IR 校验与往返、Blockly JSON 与 IR 互转、解释器、JS 与 Python 代码生成、三模式 sync-threeway、积木 i18n、编辑器状态库、Studio API、代码示例
统计内核	7	描述统计、假设检验、多重比较校正、效应量、功效分析、特殊函数、结果叙述转写

主题	文件数	覆盖内容
平台核心	7 (+2)	计算服务与 WGSL、GPU 计算、WASM 加载器、i18n；另有 <code>core/errors</code> 与 <code>core/validation</code> 各 1
可复现性	4	随机数、运行清单、锁文件、快照
R 运行时	3	运行时工厂、完整运行时回落、DSL
不确定度	3	Bootstrap、蒙特卡洛、MCMC 与诊断
建模、推断、I/O、插件 签名、扫描、模板	各 2	线性代数与拟合、贝叶斯比较与 ONNX、HDF5/NetCDF 等加载器、签名验证、设计点展开、学科模板
其余单文件模块	17	AI 助手意图、分块读取、数据质量、组图、血缘、模型、笔记本、打包、页面、绘图、画像、报告、信号、SQL、状态库、单位

值得注意的测试技术：

- **模块级缓存重置。**像 WASM 加载器这类带模块级状态的模块，每个用例前用 `resetModules` 加 `doMock` 重置，再动态导入新实例，避免用例之间互相污染。
- **FakeWorker 驱动真实运行时。**插件 Worker 的 RPC 用假 Worker 承载真实的运行时实现，在 Node 里完成端到端往返，无需浏览器。
- **严格模式的陷阱有专门说明。**`new Function` 在严格模式下无法同时使用 `"use strict"` 指令与默认参数值，遗留沙箱为此改用普通参数并显式传 `undefined`——这类“看起来是风格问题、实则是正确性问题”的点都沉淀在测试注释里。
- **回归用例制度化。**已修复的插件缺陷沉淀为 `pluginBugfixes` 等回归用例，i18n 键值在测试中校验齐备，避免复发。

端到端测试 (Playwright-core)

每套脚本各自启动 `vite preview` 生产预览，驱动无头浏览器，断言真实像素与零控制台错误：

套件	端口	覆盖内容
smoke-test	4173	启动、自动加载插件、自动建项目、响应式参数、最近项目恢复
verify-ui	4173	改版后的功能回归与布局几何：尺寸、主题切换、画布可见性、插件列表
verify-fixes	4177	自动适配、统一等宽字体、Run 控件开关、顶栏分簇
verify-3d	4199	宿主 Three.js 场景中的三维点云
verify-plugins	4198	三维与二维视口可见性互斥、等值线渲染涡旋场、散点与龙卷风示例
verify-webgpu	4289	两阶段： <code>tests/e2e/webgpu.html</code> 取真实 WebGPU 设备驱动 Rust/WASM 核心做数值校验；再在无头 Edge (SwiftShader) 中校验 GPU 与 CPU 一致性 (约 $2e-6$ 内) 与粒子插件的 GPU 提示
verify-block-mode	4174	积木模式：模式切换、编译、运行、积木到代码同步
verify-code-mode	4175	Monaco 加 Pyodide：运行 Python、变量面板、控制台、绘图画布
verify-lang-modes	4176	R / JS 代码模式与跨模式切换、R→JS 互译、Flow $\rightleftharpoons$ Block $\rightleftharpoons$ Code 无损循环
verify-ai-samples	4211	样例对话框逐个加载四个 AI 训练样本并捕获提示文案，使解析失败可观测而非靠推断
verify-ai-training	4199	AI 训练器：参数面板、样例加载、TF.js 真实训练与损失曲线、模型切换重置、决策边界、MNIST CNN
verify-research	4173	科研工具链路：启动器 → 运行记录 → 血缘 → 图表工作台 → 补充材料打包 → 笔记本
verify-site	—	合并站点完整性：校验 8 条规范路径 (站点契约)

共享 harness。所有套件都走 `scripts/_harness.mjs`，把过去散落在各脚本里、只在某台机器上成立的三件事统一为“从环境解析并带合理默认值”：浏览器可执行路径（按平台列出候选，Edge 优先、Chrome 兜底）、截图目录（系统临时目录）、预览端口（被占用时自动另取）。harness 同时保证清理，断言失败不会泄漏预览服务或浏览器进程。

链路与**链外**。 `npm run test:e2e` 串联 11 套： `smoke-test`、 `verify-ui`、 `verify-fixes`、 `verify-3d`、 `verify-plugins`、 `verify-webgpu`、 `verify-block-mode`、 `verify-code-mode`、 `verify-ai-samples`、 `verify-ai-training`、 `verify-research`。 `verify-lang-modes` 与 `verify-site` 不在该链内——前者依赖 Monaco 与 Pyodide 的完整加载（耗时较长），后者校验的是部署产物而非开发预览，因此各有独立入口（`npm run verify:site`）。

贡献指引要求：新插件或新功能必须附带端到端检查。

性能基准与体积预算

性能回归靠**基线对比**而非绝对阈值，且基线按环境分文件：

- `src/core/bench/` 的 `runAllBenchmarks()` 执行导入 / GPU 内核 / 渲染帧率 / 内存四类关键路径套件，返回含环境信息（Node 版本、平台、架构、CPU 型号、日期）的结构化结果。
- `scripts/bench-run.mjs` 在 Node 中无头运行并写入 `bench-results.json`；`scripts/bench-compare.mjs` 与基线逐指标比对——`bench/baseline.json` 面向开发机，`bench/baseline.<platform>-<arch>.json`（仓库中已有 `baseline.linux-x64.json`）面向 CI，因为共享 Runner 上的数字与开发桌面没有可比性；`scripts/bench-report.mjs` 产出单文件 HTML 报告。
- 新环境首次运行没有基线时，`bench-compare.mjs` 记录一份并以 0 退出，由 bench 工作流把该文件提交回仓库，供后续运行真正设门禁。另有两类**硬件噪声守卫**会重录基线而非误报失败（重录文件**故意不提交**，避免"每次运行都产生一次提交"的噪声）：
① Runner 报告的 CPU 型号或 Node 版本与基线不一致；
② CPU/Node 字符串相同、但吞吐指标中 $\geq 75\%$ 同向回归且内存指标全部持平——这是同型号 CPU 被换到不同宿主机（微码/固件/负载不同）的典型签名；选择性或混合方向的真实代码回归不会命中该守卫，仍会正常失败。
- 体积侧由 `scripts/build-budget.mjs` 分析构建产物，对照 `budget-baseline.json` 做门禁（FR-06）。

持续集成（5 条工作流）

工作流	触发	关键步骤与门禁
ci	push 到 main、所有 PR	引用元数据检查 (FR-10) → <code>tsc --noEmit</code> → <code>npm test</code> → 生成 WASM 存根 → <code>build:web</code> → 性能预算 (FR-06)。任一步失败即红
bench	push 到 main、发布 Release	<code>npm run bench:ci</code> （跑基准 + 按环境基线比对）→ 首次运行提交环境基线 → 结果写入 job summary 并上传 artifact；发布时作为附件挂到 Release (FR-23)

工作流	触发	关键步骤与门禁
security	push 到 main、PR、每周一 03:00 UTC、手动	三个并行 job: OSSF Scorecards (SARIF 产物, 留存 30 天)、SBOM 生成 (CycloneDX 1.5, 由无依赖 Node 脚本从 <code>package-lock.json</code> 生成, 留存 90 天)、 <code>npm audit</code> (high / critical 阻断合并 , 带需写明理由与后续 issue 的 <code>AUDIT_ALLOWLIST</code> 豁免机制, 豁免项降级为警告)
release	推送 <code>v*</code> 标签	build: 生成 WASM 存根 → <code>build:web</code> → 生成 <code>CITATION.cff</code> → 打包 dist zip; archive: Zenodo 存缴并铸造 DOI, 未配置 <code>ZENODO_TOKEN</code> 时显式报错并给出配置指引 , 而不是静默跳过归档; release: 发布 GitHub Release, 附 zip、 <code>CITATION.cff</code> 与 DOI
deploy	push 到 main、手动	装 Rust (<code>wasm32-unknown-unknown</code> 目标) 与 <code>wasm-bindgen-cli 0.2.127</code> → 编译真实 WASM → 构建工作台 <code>app/</code> 与官网根站 → <code>deploy:merge</code> 合并产物 → <code>verify-site</code> 校验 8 条规范路径 → 发布 GitHub Pages

CI 里没有 Rust 工具链 (deploy 除外) : 其余工作流的 `dist` 由 `scripts/make-wasm-stub.mjs` 生成模块存根, 因为 `vite build` 会把 `src/core/wasm.ts` 中的动态导入切成独立 chunk, 构建期只需要该 chunk 存在、不需要真实内核。

构建链自带门禁: `npm run build` = `build:wasm && tsc --noEmit && vite build && build-docs`, 任一步失败即中止; 纯前端场景可用 `npm run build:web` (跳过 WASM 编译) 。

五 工程脚本

`scripts/` 下 40 个脚本按用途分为六类:

类别	脚本
端到端与 harness	<code>_harness.mjs</code> (共享 harness)、 <code>smoke-test.mjs</code> 、 <code>verify-*.mjs</code> (12 个)、 <code>_line-check.mjs</code> (目检 Monaco 当前行与选区实际渲染色)
性能与预算	<code>bench-run.mjs</code> 、 <code>bench-compare.mjs</code> 、 <code>bench-report.mjs</code> 、 <code>build-budget.mjs</code>
构建与部署	<code>build-wasm.mjs</code> 、 <code>make-wasm-stub.mjs</code> 、 <code>build-docs.mjs</code> 、 <code>merge-deploy.mjs</code> 、 <code>gen-sw-precache.mjs</code> 、 <code>copy-pyodide.mjs</code> 、 <code>copy-webr.mjs</code> 、 <code>vendor-blockly-media.mjs</code>
资产与示例生成	<code>make-example-data.mjs</code> 、 <code>make-example-projects.mjs</code> 、 <code>gen-ai-examples.mjs</code> (种子化 PRNG, 产出确定、可提交、可被学习者检查的 CSV)

类别	脚本
合规与签名	<code>sign-cspkg.mjs</code> 、 <code>gen-sbom.mjs</code> 、 <code>gen-citation-cff.mjs</code> 、 <code>check-citation.mjs</code>
技术文档排版	<code>build-tech-docs.mjs</code> (md → 分页 HTML → A4 PDF)、 <code>build-tech-summary.mjs</code> (从分篇汇编合集, 使合集不可能与分篇漂移)

其中 `gen-citation-cff.mjs` 与 `check-citation.mjs` 一写一校, 共同维护 `CITATION.cff`; `gen-sw-precache.mjs` 生成 Service Worker 预缓存清单, 使离线可用范围与构建产物严格一致。

技术文档排版链路。 `docs/technical/` 下的九份文档由 `node scripts/build-tech-docs.mjs` 重建: Markdown 经内置的子集解析器转成区块, mermaid 围栏在构建期用无头浏览器渲染为内联 SVG (产物不含任何客户端脚本, PDF 里是可缩放的矢量图), 随后在真实页面尺寸下**逐块测量高度**并按页高装填——固定高度的 `.page` 配 `overflow: hidden`, 靠目测平衡内容会在任何一节变长时把内容裁掉, 因此分页必须由测量驱动。测量结果同时回填目录页码, 并在生成后逐页复检"内容是否越过下边距", 发现溢出即以缩小后的可用高度重排。技术总结 (`Ergalics Studio.md`) 则由 `build-tech-summary.mjs` 从八篇分篇源码汇编, 因此不可能出现"分篇已改、合集还写着旧数字"的情况。

六 质量原则

1. **纯函数优先。**编译器、统计内核、绘图与组图引擎、代码生成与 IR 转换全部是纯函数, 可在 Node 环境直接测试; 血缘图的分层布局同样无 React / store / DOM 依赖。
2. **副作用收口。**区块系统里只有渲染桥接一个带副作用的模块, 执行器异步但无副作用; 研究工具页的胶水层 (`researchUi.ts`) 也被刻意写成无 React 依赖的薄层, 以便单独测试。
3. **数值一致性兜底。**每个 GPU 内核都有数学一致的 CPU 实现, 单测与端到端两层校验二者一致——这是"自动降级"能够被信任的前提。
4. **边界显式化。**边界输入抛结构化错误、空数据启动有守卫提示、环境缺失如实自检报告, 而不是静默失败或静默降级; 可复现性有专门的门禁文档告知。
5. **不静默跳过。**需要归档就必须归档 (Zenodo 未配置即报错), 需要门禁就必须门禁 (审计豁免要写明理由), 第一次运行没有基线就记录并提交——把"没跑成"和"跑过了"区分清楚。

让每一次科学计算，

都发生在离思想最近的地方

Ergalics Studio 把复杂度收进浏览器：安装、配置与数据搬运都不该出现在科研的路上。打开即可计算，数据留在本机，四种工作模式共享同一份数据——一张图、一条流程与一行代码，都是同一次工作的不同表达。

工具越安静，思想就越清楚。我们做的每一处取舍，都为了把注意力还给问题本身。

作者 [SnowLeopard-io](#) github.com/SnowLeopard-io